```python
import random
import math

# ---------------- PARAMETERS ----------------
GRID_SIZE = 20
SENSING_RADIUS = 3
NUM_SENSORS = 5
POPULATION_SIZE = 20
GENERATIONS = 10
MUTATION_RATE = 0.1

# ---------------- FITNESS FUNCTION ----------------
def fitness(chromosome):
    covered = 0
    total_points = GRID_SIZE * GRID_SIZE
    for x in range(GRID_SIZE):
        for y in range(GRID_SIZE):
            # Check whether this grid point is covered
            for sx, sy in chromosome:
                distance = math.sqrt((x - sx) ** 2 + (y - sy) ** 2)
                if distance <= SENSING_RADIUS:
                    covered += 1
                    break
    return (covered / total_points) * 100

# ---------------- CREATE CHROMOSOME ----------------
def create_chromosome():
    chromosome = []
    for _ in range(NUM_SENSORS):
        x = random.randint(0, GRID_SIZE - 1)
        y = random.randint(0, GRID_SIZE - 1)
        chromosome.append((x, y))
    return chromosome

# ---------------- INITIAL POPULATION ----------------
population = [create_chromosome() for _ in range(POPULATION_SIZE)]

# ---------------- GENETIC ALGORITHM ----------------
for generation in range(GENERATIONS):
    # Calculate fitness (Sorts descending: best fitness first)
```

```
    population.sort(key=fitness, reverse=True)

    # Best chromosome
    best = population[0]
    best_fitness = fitness(best)
    print(f"Generation {generation + 1}: Coverage = {best_fitness:.2f}%")

    # Selection: keep best 50%
    survivors = population[:POPULATION_SIZE // 2]
    new_population = survivors.copy()

    # Create new chromosomes
    while len(new_population) < POPULATION_SIZE:
        # Select two parents
        parent1 = random.choice(survivors)
        parent2 = random.choice(survivors)

        # Single-point crossover
        point = random.randint(1, NUM_SENSORS - 1)
        # FIX: Changed outer parentheses to square brackets to create a
list
        child = parent1[:point] + parent2[point:]

        # Mutation
        if random.random() < MUTATION_RATE:
            sensor = random.randint(0, NUM_SENSORS - 1)
            new_x = random.randint(0, GRID_SIZE - 1)
            new_y = random.randint(0, GRID_SIZE - 1)
            child[sensor] = (new_x, new_y)

        new_population.append(child)

    population = new_population

# ---------------- FINAL RESULT ----------------
population.sort(key=fitness, reverse=True)
best_solution = population[0]
print("\nFinal Sensor Coordinates :")
print(best_solution)
print(f"Coverage Percentage: {fitness(best_solution):.2f}%")
```

Output:

```
...  Generation 1: Coverage = 31.75%
     Generation 2: Coverage = 34.00%
     Generation 3: Coverage = 34.00%
     Generation 4: Coverage = 34.00%
     Generation 5: Coverage = 34.00%
     Generation 6: Coverage = 34.00%
     Generation 7: Coverage = 34.00%
     Generation 8: Coverage = 34.00%
     Generation 9: Coverage = 34.00%
     Generation 10: Coverage = 34.00%

     Final Sensor Coordinates :
     [(10, 7), (5, 14), (13, 2), (10, 14), (17, 5)]
     Coverage Percentage: 34.75%
```